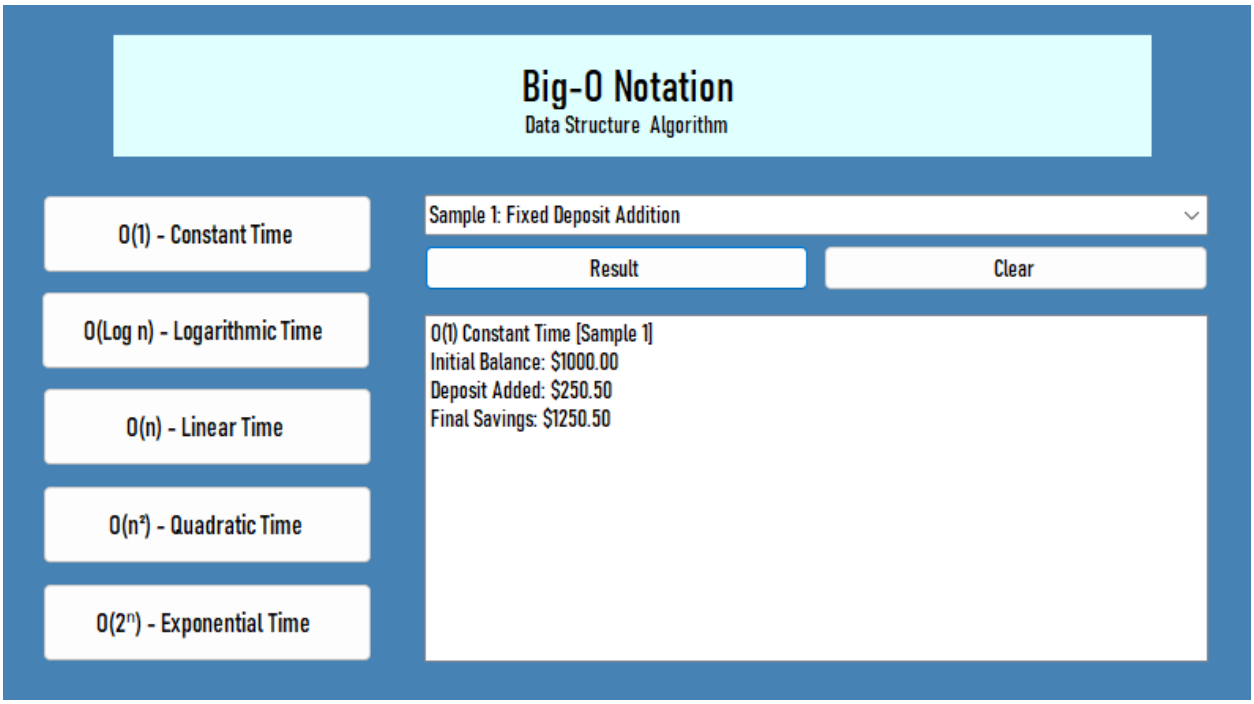
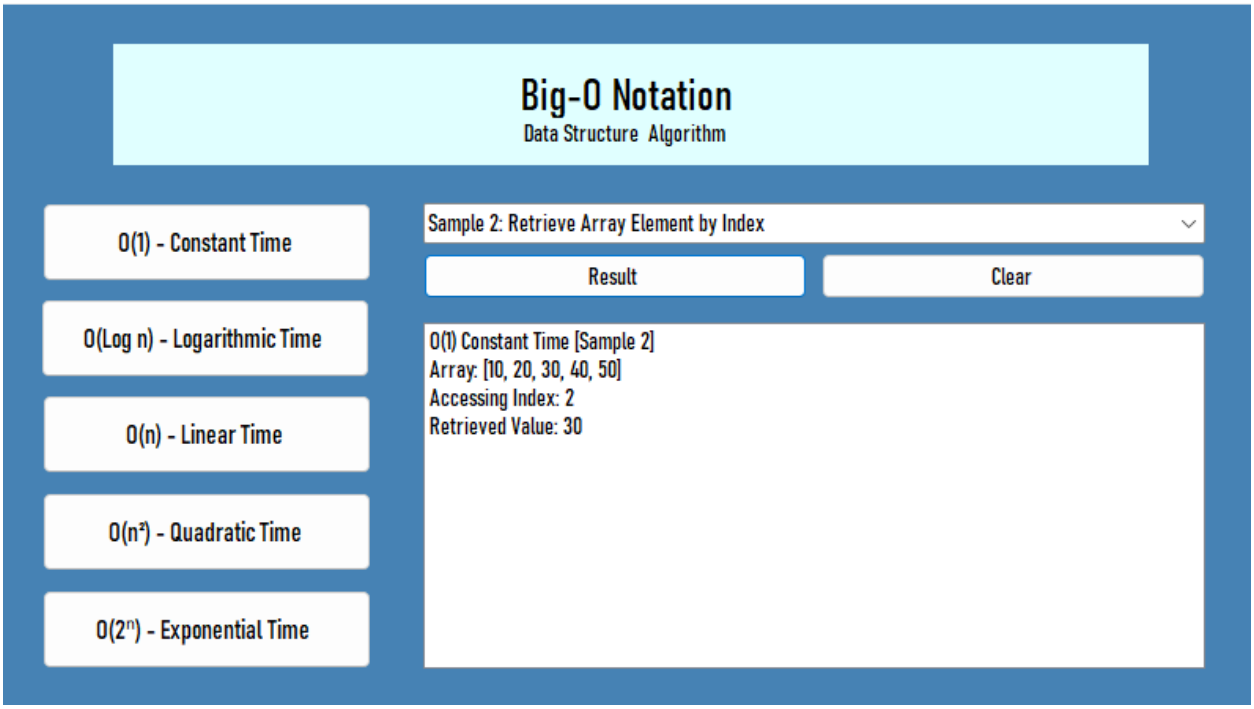
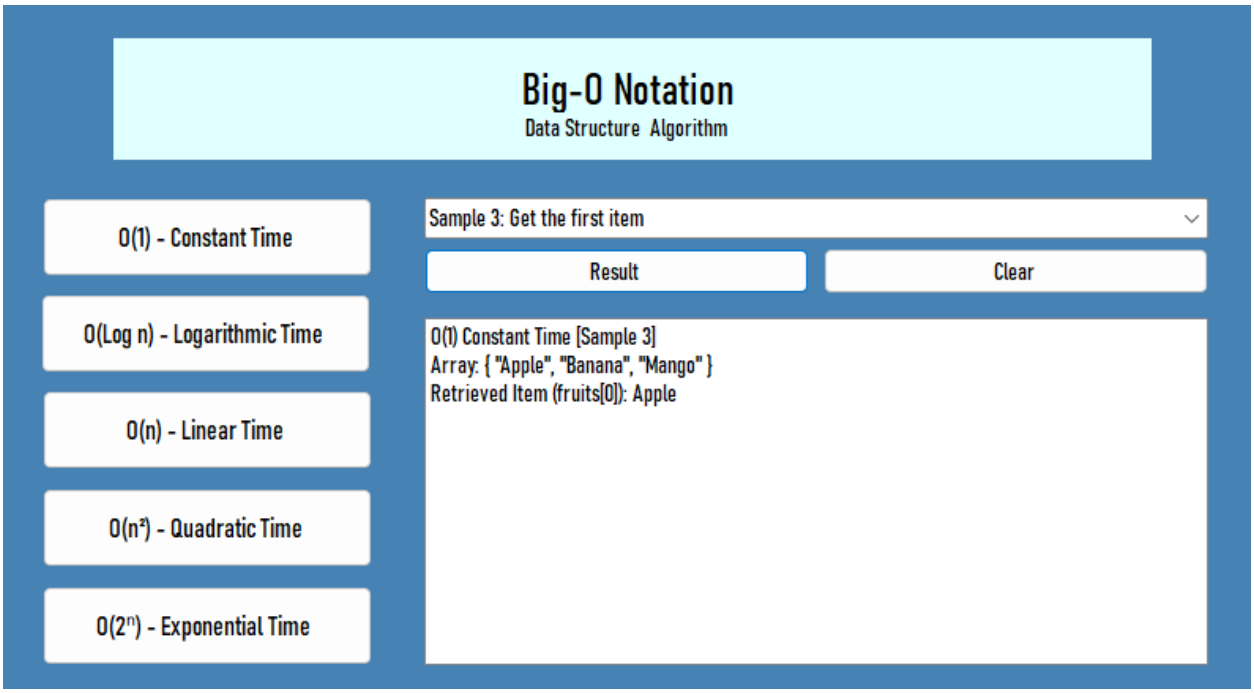
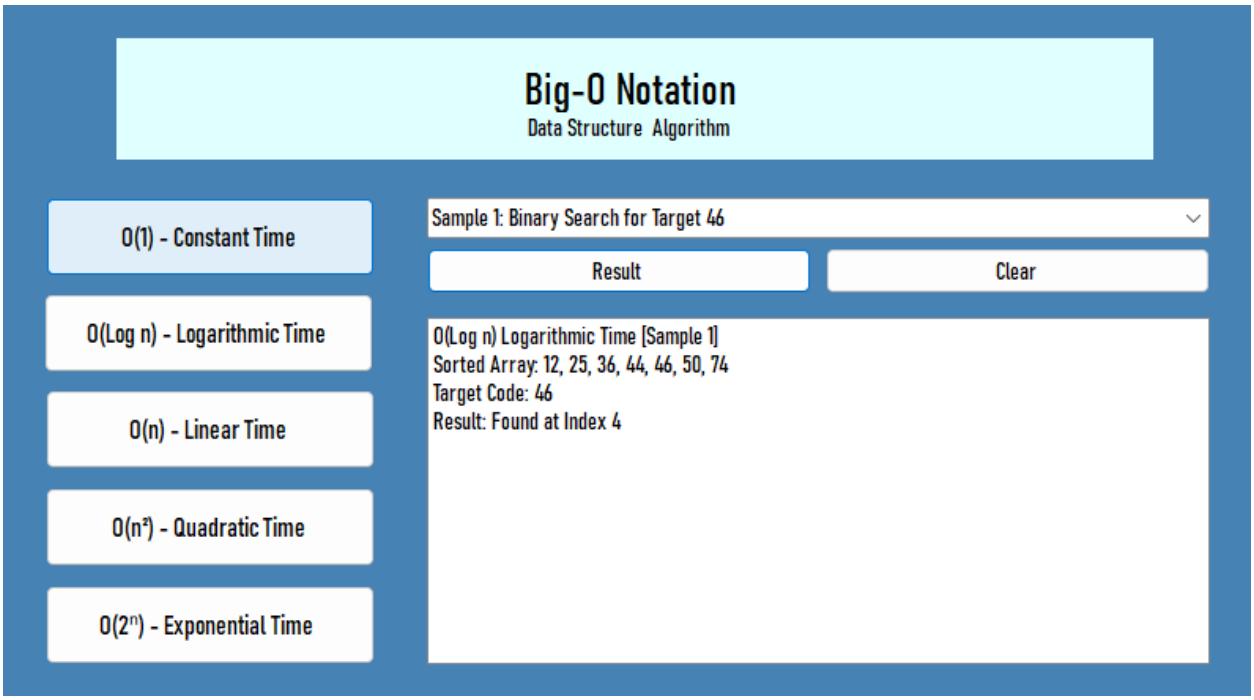
Constant Time

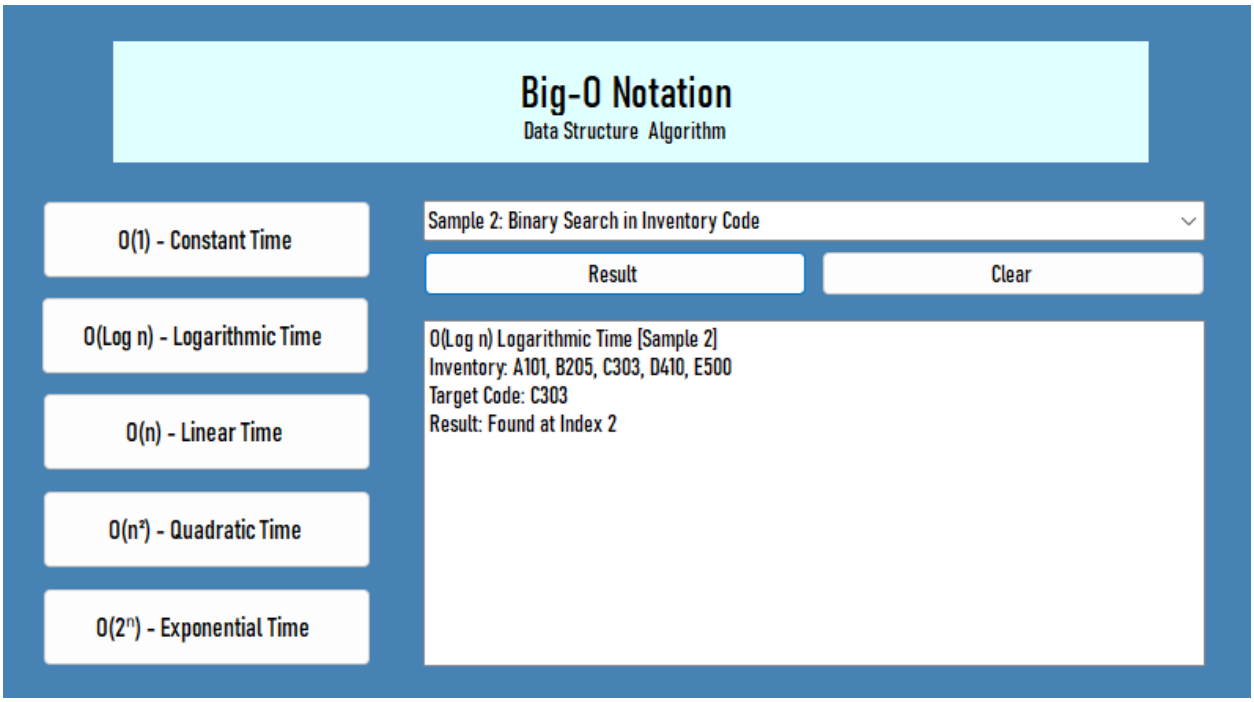






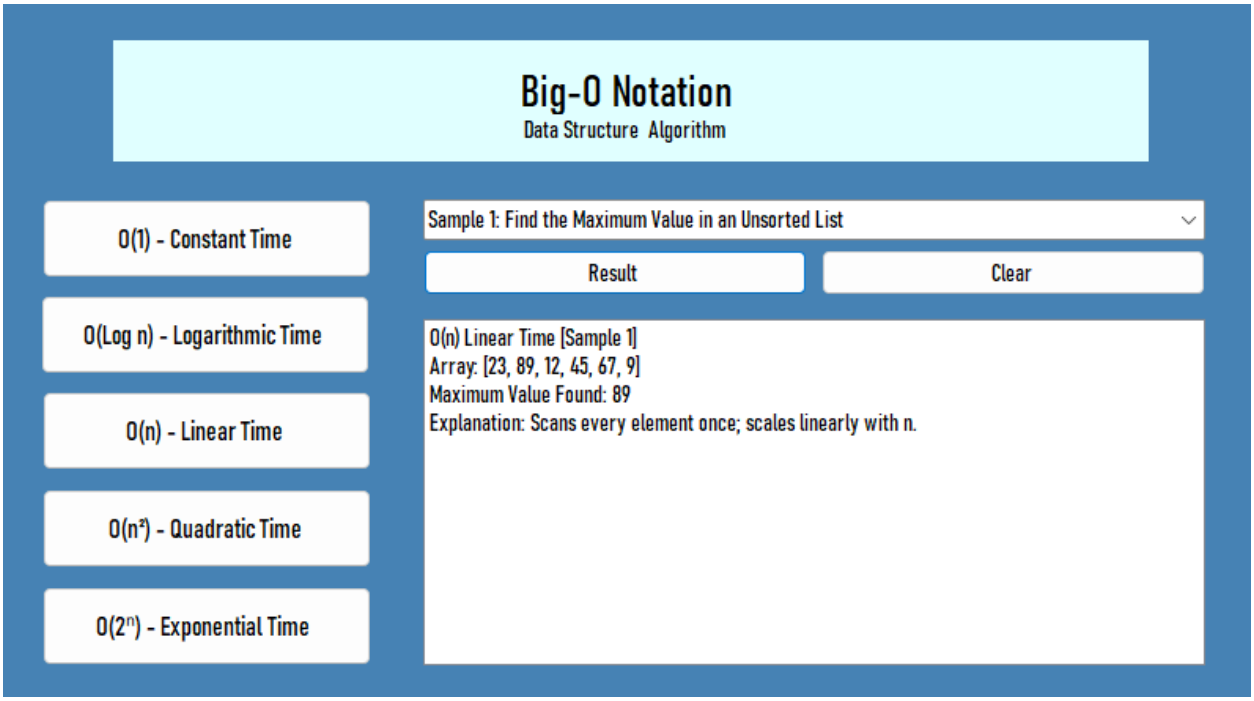
Logarithmic Time

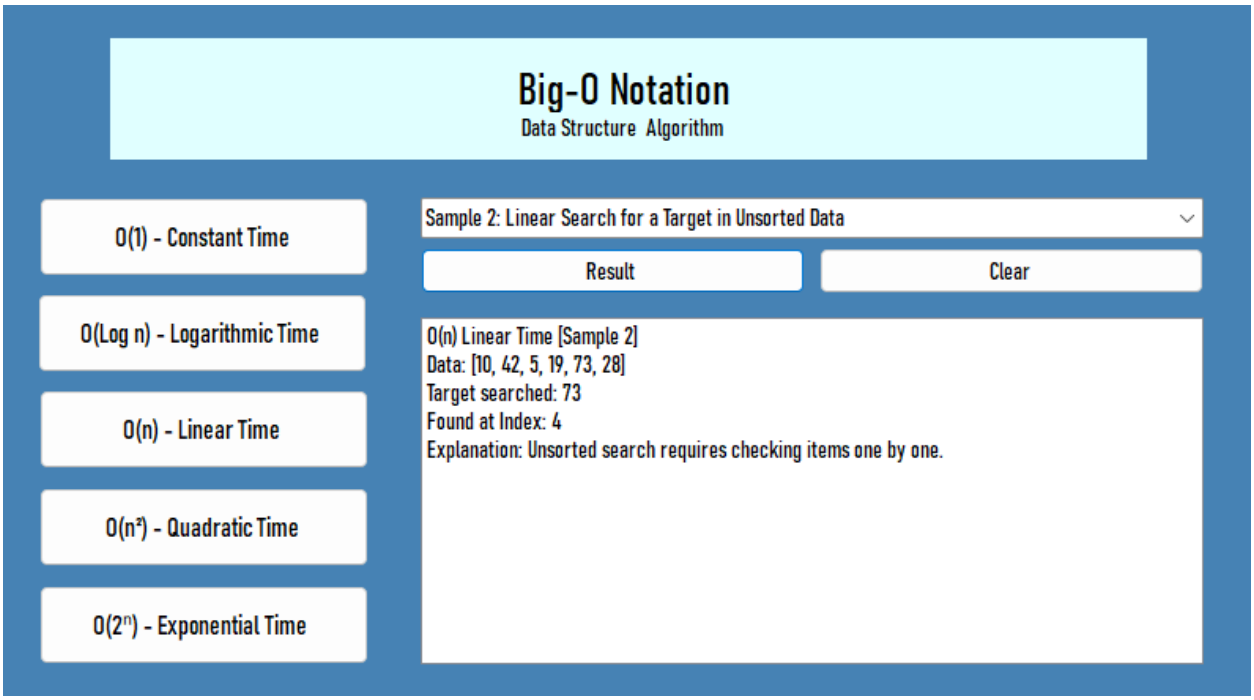

Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 1: Binary Search for Target 46
Result
Clear
O(Log n) Logarithmic Time [Sample 1]
Sorted Array: 12, 25, 36, 44, 46, 50, 74
Target Code: 46
Result: Found at Index 4

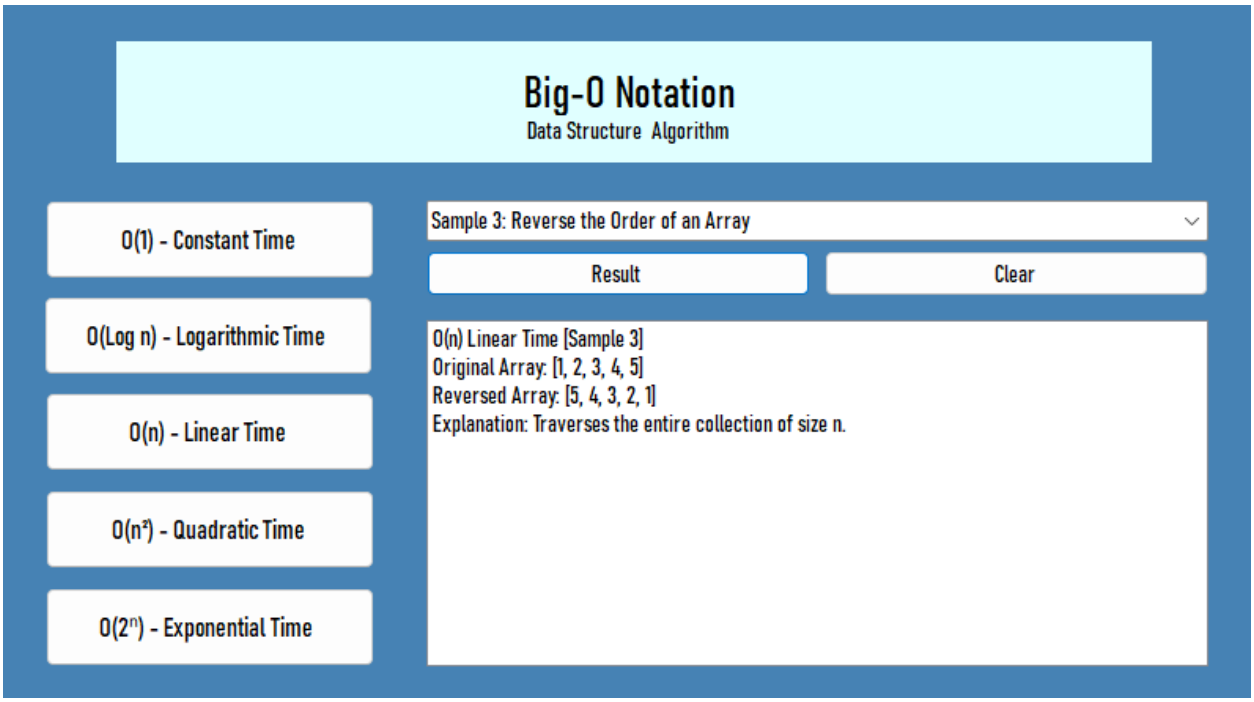

Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 2: Binary Search in Inventory Code
Result
Clear
O(Log n) Logarithmic Time [Sample 2]
Inventory: A101, B205, C303, D410, E500
Target Code: C303
Result: Found at Index 2


Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 3: Halving Numbers Step Count
Result
Clear
O(Log n) Logarithmic Time [Sample 3]
Starting Value (n): 32
Total Halving Steps to reach 1: 5

Linear Time

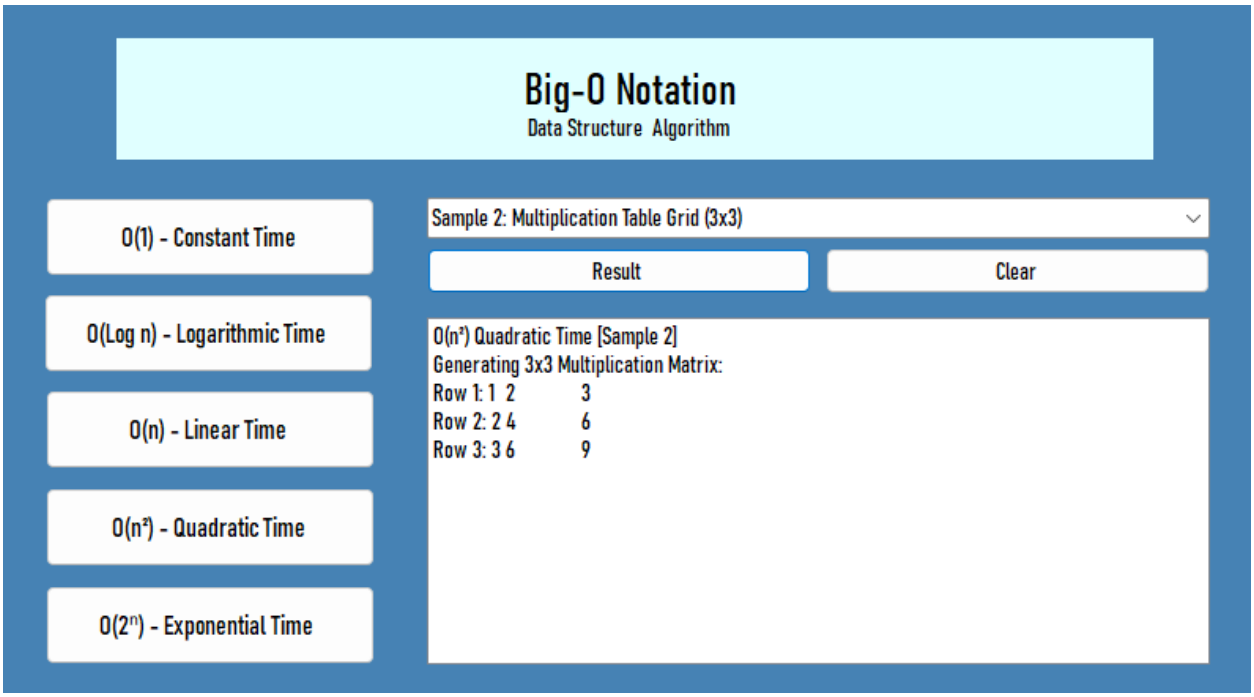

Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 1: Find the Maximum Value in an Unsorted List
Result
Clear
O(n) Linear Time [Sample 1]
Array: [23, 89, 12, 45, 67, 9]
Maximum Value Found: 89
Explanation: Scans every element once; scales linearly with n.

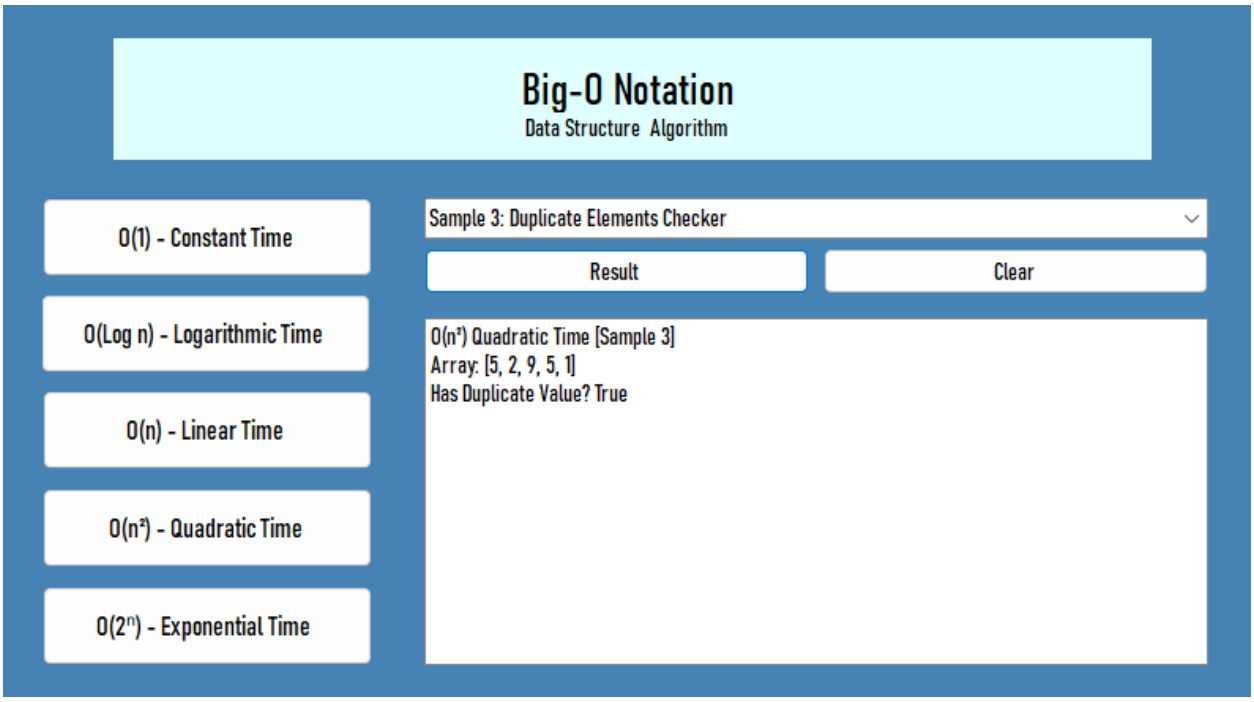

Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 2: Linear Search for a Target in Unsorted Data
Result
Clear
O(n) Linear Time [Sample 2]
Data: [10, 42, 5, 19, 73, 28]
Target searched: 73
Found at Index: 4
Explanation: Unsorted search requires checking items one by one.


Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 3: Reverse the Order of an Array
Result
Clear
O(n) Linear Time [Sample 3]
Original Array: [1, 2, 3, 4, 5]
Reversed Array: [5, 4, 3, 2, 1]
Explanation: Traverses the entire collection of size n.

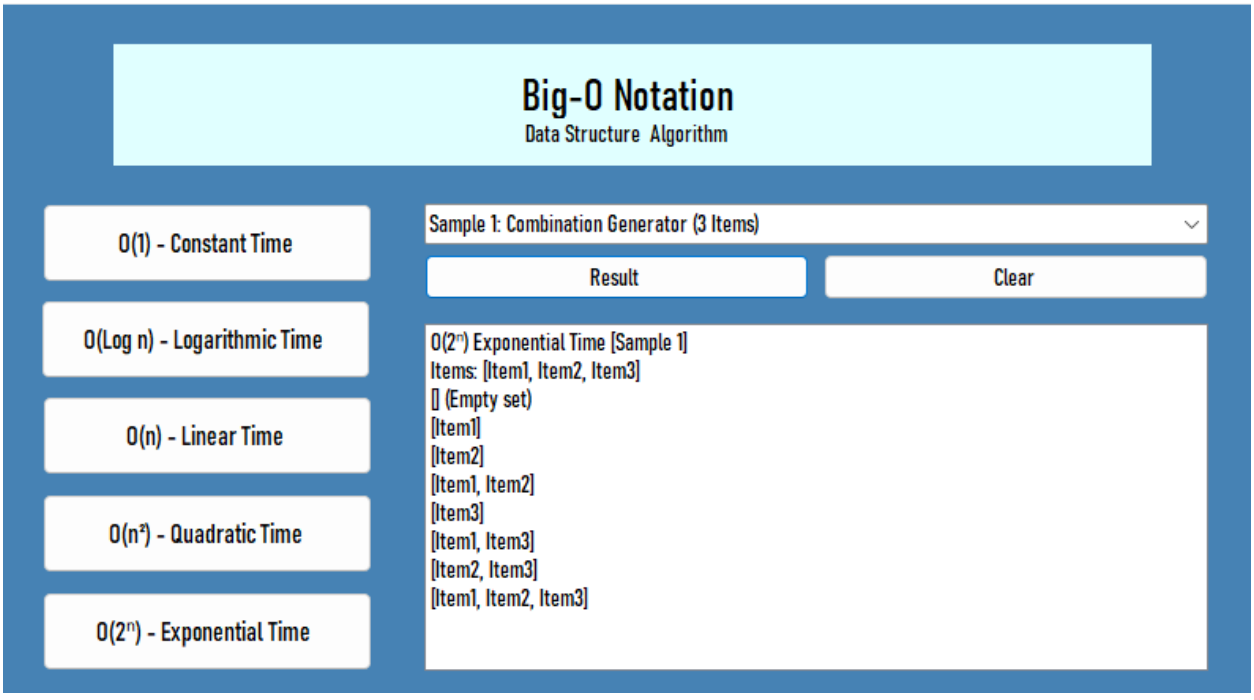
Quadratic Time

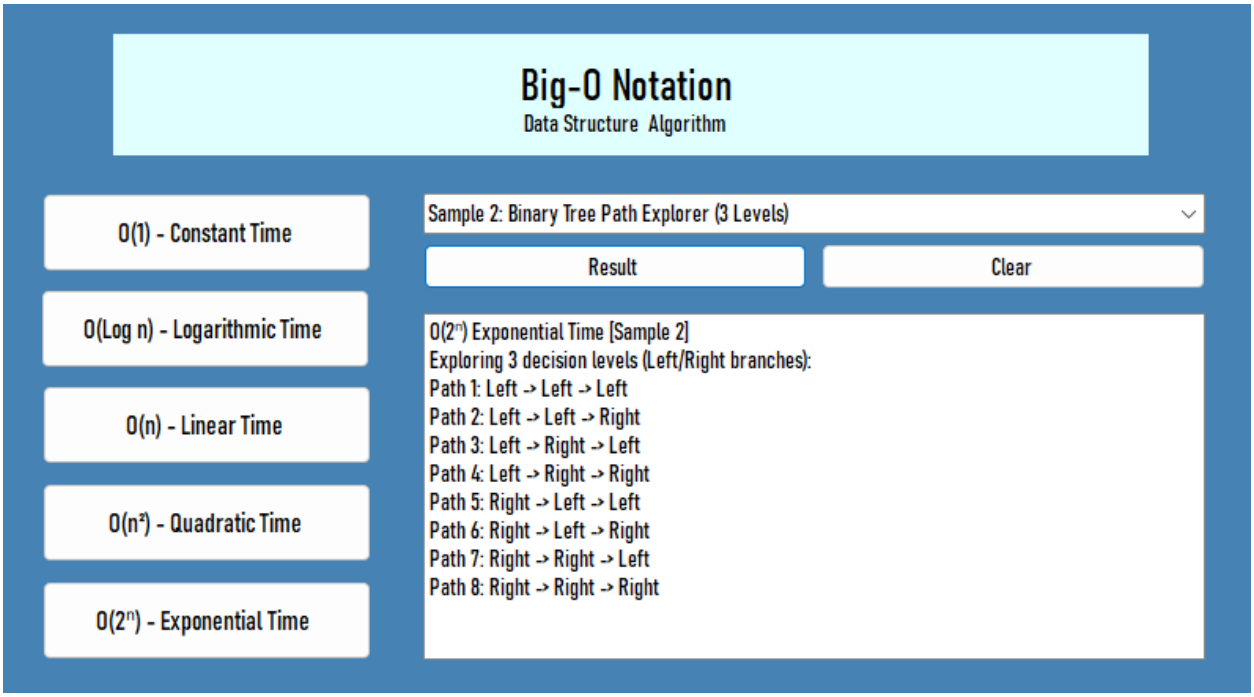


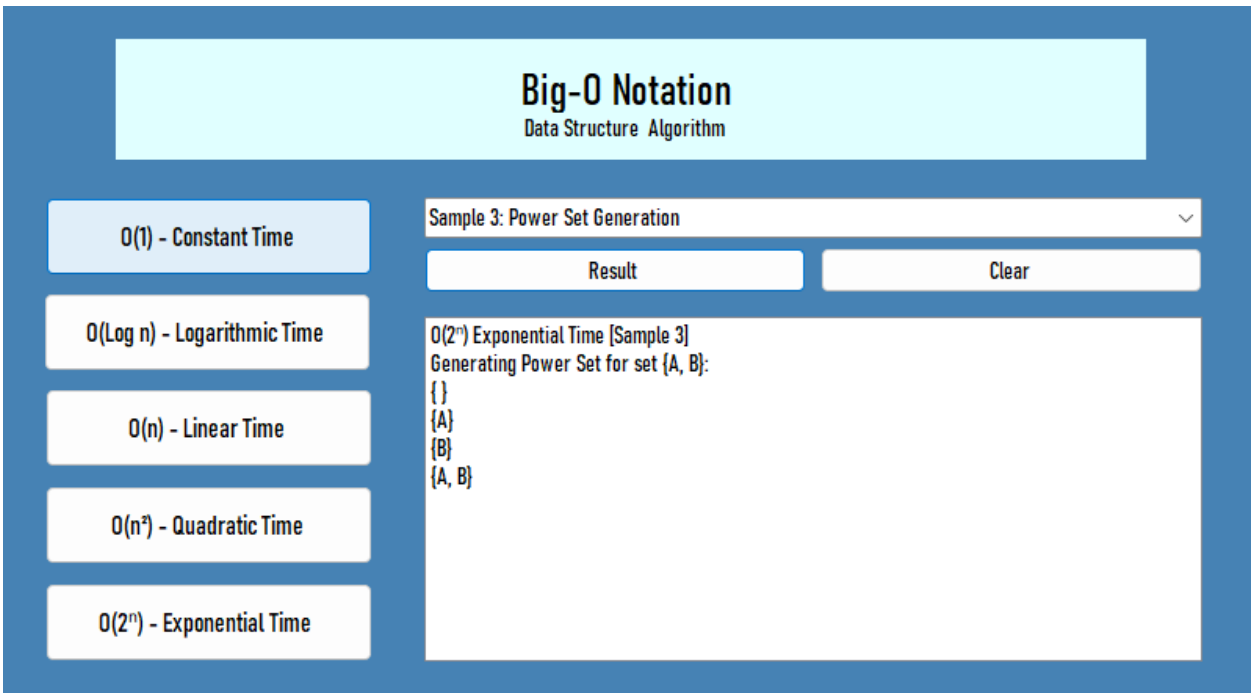




Exponential Time




Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 2: Binary Tree Path Explorer (3 Levels)
Result
Clear
O(2ⁿ) Exponential Time [Sample 2]
Exploring 3 decision levels (Left/Right branches):
Path 1: Left -> Left -> Left
Path 2: Left -> Left -> Right
Path 3: Left -> Right -> Left
Path 4: Left -> Right -> Right
Path 5: Right -> Left -> Left
Path 6: Right -> Left -> Right
Path 7: Right -> Right -> Left
Path 8: Right -> Right -> Right


Big-O Notation
Data Structure Algorithm
O(1) - Constant Time
O(Log n) - Logarithmic Time
O(n) - Linear Time
O(n²) - Quadratic Time
O(2ⁿ) - Exponential Time
Sample 3: Power Set Generation
Result
Clear
O(2ⁿ) Exponential Time [Sample 3]
Generating Power Set for set {A, B}:
{ }
{A}
{B}
{A, B}